

GRONINGENPHP MEETUP · 10 SEPTEMBER 2026

Spec-Driven Development.

From vibe-coding to integrating AI coding assistants into the
Software Development lifecycle.

Agenda

- 01** The bottleneck moved
- 02** What SDD is, and its caveats
- 03** What actually works
- 04** This is not hypothetical
- 05** The ask

01

The bottleneck moved

THE ROOM'S OWN EXPERIENCE

Adoption is settled. Trust is not.

84%

use or plan to use AI tools in
their development process

46%

do **not** trust the accuracy of
what those tools produce,
against 31% a year earlier

66%

name the same top
frustration: answers almost
right, but not quite

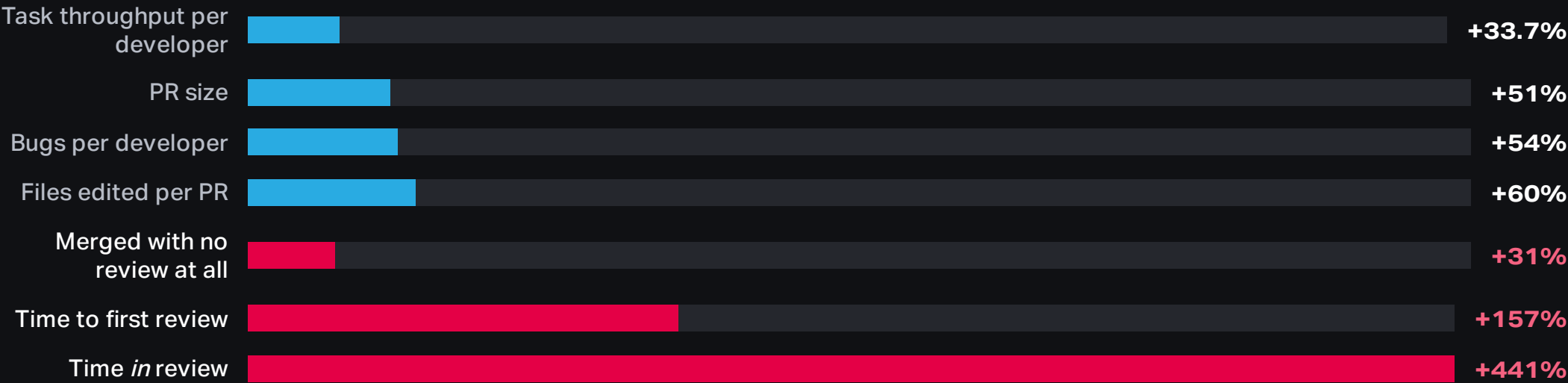
45%

say debugging AI-written
code takes longer than
writing it themselves

Stack Overflow Developer Survey 2025, 49,000 respondents. Nobody here is arguing about whether we use agents. That argument is over. The argument is about what happens next.

HIGH-ADOPTION TEAMS AGAINST LOW, OVER TWO YEARS

Review is where the work went.



Faros AI, *AI Engineering Report 2026: The Acceleration Whiplash*, April 2026.
22,000 developers, 4,000 teams, two years. Vendor telemetry, correlational, self-
selected adoption: directional, not causal. Not plotted, and worse: code churn
+861%, incidents per PR +242.7%.

DORA, 2025 AND 2026

AI amplifies what a team already is.

THE HINGE

Where testing, version control discipline and fast feedback are strong, agents compound the strength. Where they are weak, more change volume produces instability. AI has a **negative** relationship with delivery stability.

The 2026 ROI follow-up adds a **J-curve**: value goes negative before it goes positive, and positive return is not guaranteed.

So the question is not whether to use agents. **It is what we amplify them with.**

02

What it is, and its caveats

Settle the problem, the approach, and what counts as done.

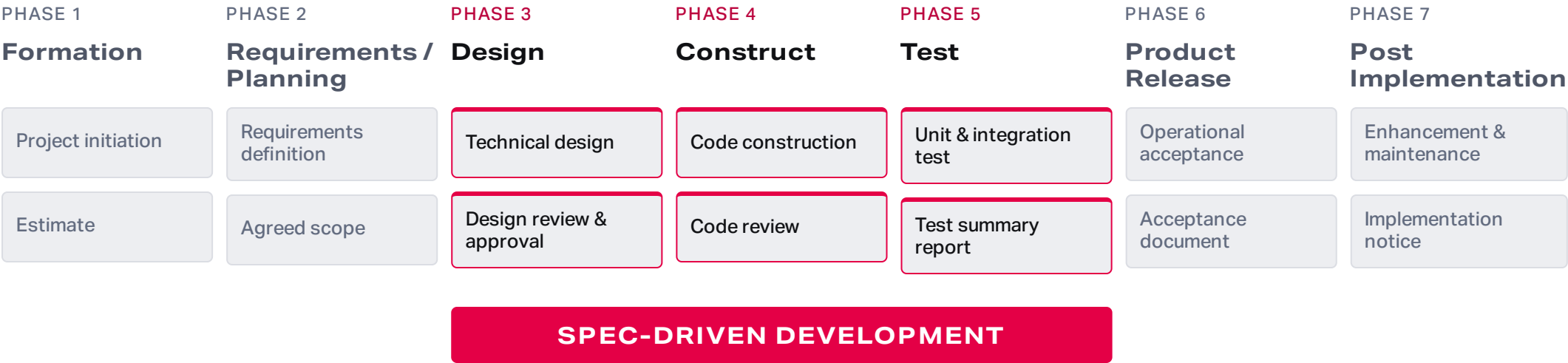
- The **spec** fixes the problem and the boundaries. The **plan** fixes which files change and in what order. The **check** names the command or test that must pass.
- These are **files in the repository, not a prompt**. The agent reads them at the start of every session. A chat context does not survive one.
- Sign-off moves from the diff, where changing your mind costs a rewrite, to **the plan, where it costs a paragraph**.

Four artifacts.

- 01 Constitution:** the rules and patterns that do not change per feature. How we commit, what needs a human yes. Written once, read every session.
- 02 Spec:** what and why. The problem, the decision, the boundaries. No implementation.
- 03 Plan:** how, grounded in *the* codebase. Which files, which order, and what check proves it worked.
- 04 Tasks:** units an agent can execute and a human can tick off.

THE LIFECYCLE YOU ALREADY RUN

Three phases out of seven.



HOW SDD ACTUALLY RUNS

The first four steps happen before any code exists.

BEFORE CODE: YOU DECIDE

01

INTENT

what problem

02

SPEC

what and why

03

PLAN

how, in this
codebase

04

TASKS

units of work

AFTER CODE: AGENT, GATE, HUMAN

05

IMPLEMENT

the agent writes

06

VERIFY

the hook
gates it

07

REVIEW

a human
reads it

FIVE INDEPENDENT TEAMS, FIVE VOCABULARIES

Everyone independently invented the same four artifacts.

FRAMEWORK	RULES THAT NEVER CHANGE	WHAT & WHY	HOW, IN THIS CODEBASE	UNITS OF WORK
GitHub Spec Kit	constitution.md	spec.md	plan.md	tasks.md
AWS Kiro	steering files	requirements.md	design.md	tasks.md
BMAD-METHOD	agent personas	PRD	architecture	story files
OpenSpec	project conventions	proposal	design	tasks
Superpowers	CLAUDE.md + skills	brainstorming	writing-plans	executing-plans

SCOTT LOGIC REBUILT 1,000 DELETED LINES, TWICE, AND MEASURED IT

The strongest case against it.

MEASURE	SPEC KIT, FEATURE 1	SPEC KIT, FEATURE 2	ITERATIVE PROMPTING
Agent run time	33m30	23m30	8 min
Lines of code produced	689	~300	1,000
Lines of markdown produced	2,577	2,262	0
Human review	3.5 hours	~2 hours	24 min

Roughly ten times slower end to end, and it still shipped a small, obvious bug, with no process for folding the fix back into the spec. Colin Eberhardt, Scott Logic, November 2025.

THE CRITICS

We've reinvented Big Design Up Front. We just replaced Word documents with Markdown and project managers with LLMs.

Nils Kjellman, QueensLab. *Your Spec Driven Workflow Is Just Waterfall With Extra Steps*, November 2025.

AND THE FIELD CONCEDED, IN PUBLIC

- **Agent OS v3.0** deleted its own spec, task and orchestration layer in January 2026, and now defers to plan mode.
- **Tessl** raised \$125M on spec-as-source in 2024. Its homepage headline today: "Skills are the new code."
- **Instil** published "Spec-driven development is dead, long live plan mode" in June 2026.

buildermethods/agent-os v3 release notes, January 2026 · tessl.io, with the raise reported by TechCrunch, November 2024 · instil.co, 19 June 2026.

THE OBJECTION NOBODY ANSWERED

Generate code from a spec and your compiler is non-deterministic.

Run the same spec twice, get two implementations. Diffs, bisections and upgrade paths all degrade.

Most of the value of a spec comes from being forced to think about the problem before you start. Beyond that point, ceremony is cost.

What is durable is not the framework. It is two things: **getting the design and the plan right before code exists**, grounded in what is already there, and **making the rules enforceable**.

03

What actually works

Prose does not enforce anything.

- **ETH Zurich:** 438 tasks, four agents. Repository context files do not generally improve success, and add over 20% inference cost. Instructions were followed; they just did not help.
- **1,650 Claude Code sessions:** no file-structure variable had a detectable effect. What predicted compliance was position in the session: roughly **5.6% lower odds per additional function generated**.
- **100 open-source projects:** 62% show “lint leakage”: rules written as polite prose that a linter should have enforced.

"TO BLOCK AN ACTION REGARDLESS OF WHAT CLAUDE DECIDES, USE A PRETOOLUSE HOOK"

So enforcement has to be deterministic.

HOOK	FIRES	BLOCKS?	THE JOB IT DOES
PreToolUse	before a tool runs	Yes	the only real gate: guard spec files, migrations, production systems
PostToolUse	after a tool succeeds	feeds back	run the sensors: format, typecheck, targeted tests
Stop	agent tries to finish	Yes	a red suite means it cannot declare itself done
PreCompact	before context is squeezed	Yes	persist decisions before the window eats them

Guides prevent. Sensors detect. You need both.



86,156 TEST FILES FROM 33,596 AGENT-AUTHORED PULL REQUESTS, FIVE AGENTS

“The agent wrote tests” is not an answer.

- **80.2% carry weak or no oracle signal**: they execute the code without asserting on its behaviour. Agents generate test *structure* far more reliably than oracle *logic*.
- Agent tests read as **observational feedback**: value-revealing prints outnumber assertions, and prompting for more tests did not move outcomes.
- In fairness, they are better at **edge-case variety** (0.62 against 0.32) and worse at **flakiness** (0.41 against 0.30). They lack the environmental awareness a hermetic CI test needs.

Banik, Chowdhury and Shamim, arXiv 2606.18168, 16 June 2026 · Chen et al., arXiv 2602.07900 · Jhanglani et al., arXiv 2607.12068, 13 July 2026.

SAME AGENT, SAME BENCHMARK, OPPOSITE RESULTS

Tell it what to test. Do not teach it how to test.

THE TECHNIQUE	WHAT YOU ACTUALLY DO	WHAT HAPPENED
Impact analysis	Show it which tests cover the code it is about to change	broke 6.1% → 1.8% of changes
TDD prompting	Tell it to write the test first	broke 6.1% → 9.9% of changes
Property-based tests	Ask it to check a rule, not one example	up to +13% more solved first try
Mutation testing	Break the code on purpose, keep the tests that notice	73% of its tests kept

The first two rows are the same experiment, so those numbers compare directly. Showing the agent the map made it three times safer. Lecturing it about method made it worse than saying nothing. [arXiv 2603.17973](#) · [arXiv 2506.18315](#) · [Meta engineering](#), 30 September 2025.

TWO ANSWERS THAT DO NOT WORK

You cannot just show the agent the whole repository.

- **Context rot.** Across 18 frontier models, reliability degrades as the input grows, even on trivial retrieval. That kills “paste it all in, we have a million tokens.”
- **Embeddings mangle code.** Chunking source for vector search tears the logic apart, and the index goes stale the moment the codebase moves.

Grounding is not about *finding* code. It is about spending a fixed context budget on the right code, so that the plan is right before any code is written.

THE PLANNING PHASE, GROUNDED

A symbol graph is how a plan gets grounded.

Aider repo map

Tree-sitter plus personalised PageRank. A whole repository summarised in roughly a thousand tokens.

Serena

Language-server backed, 30+ languages. The agent navigates by symbol, the way an IDE does.

RepoGraph

+32.8% average relative improvement on SWE-bench as a plug-in module. ICLR 2025.

Codebase-Memory

Across 31 repositories: **10× fewer tokens**, 2.1× fewer tool calls, at 83% answer quality against 92%. A real trade, not a free win.

EVERY DEMO ONLINE IS GREENFIELD, BECAUSE AN EMPTY REPO HAS NO CONTEXT TO GET WRONG

Brownfield is the real job.

- 01 Traceability:** every claim in a recovered spec points back to source
- 02 Confidence marking:** claims are labelled by how well evidenced they are
- 03 Preserved gaps:** unknowns stay visible as unknowns

The failure mode is not an agent saying “I don’t know.” It is an agent confidently inventing a business rule that sounds plausible, which enters the spec, then the code, and surfaces as a bug six months later. Alongside it: silent spec-to-code drift, context explosion, and process theatre.

04

This is not hypothetical

TODAY'S SPEC

The platform should not have its own passwords.

The company has one identity provider. We were running a custom one.

THE CONSTITUTION · 144 LINES

Define the check before the change.

FOUR HOOKS, RUNNING TODAY

- A git action must name its repository
- A live ingest needs a human yes
- No custom references in commit messages
- No comments citing non-maintained documents

Rules with consequences are code.

TODAY'S PLAN

Four steps became twelve, in four passes.

- Roles, groups, and **a test that guards them**
- The identity provider
- Why the login page still appeared
- The client apps

The checks were named before step one. Each pass added what the last one learned.

THIRD PASS

The plan was wrong. We found out the same day.

Step four was meant to send people straight to the company login, by setting a value in the web app. **That value is baked in when the app is built and the file holding it is deliberately kept out of the build.** It could never have arrived.

set it in the web app



set it in the login server

Waterfall's flaw was never having a plan. It was waiting months to find out the plan was wrong.

THE END OF THE PLAN

It also says what
it could not finish.

DONE	NOT DONE, AND WHY
Company login works end to end	Connecting the real company account needs credentials we do not have yet
2,324 tests pass	Switching passwords off can only happen outside our test environment

Both were written down before the work started, not discovered at the end. A plan that names what it cannot do is one a reviewer can trust.

05

The ask

When it is worth the ceremony.

Auditability

What was decided. Not just what changed.

Governance

One file people and agents both read.

Parallelisation

Agents in parallel need contracts that do not overlap.

Not for prototypes, one-line fixes, or exploration.

NOBODY HAS MEASURED THIS PROPERLY. INCLUDING US.

One project. One sprint. Three numbers.

Review time per PR	where the cost landed
Rework rate	was the plan right
Escaped defects	what a client feels

Did the spec still match the code?



**Simple tasks need a prompt.
Complex ones need a spec.**

Writing the decision down turns a six-month bet
into an experiment you can fail in an afternoon.